AI Supplier Document Reader

An n8n automation reads supplier order confirmations and delivery notes, in PDF, Excel or scan, and turns them into ERP-ready import files. On a 498-line test set it read every line correctly, for about $0.04 of AI cost per run, and it reports every line it could not verify.

498/498	≈ $0.04	≈ 6 h
lines correct across repeated runs	AI cost per full run	of manual typing it replaces

BUILT WITH	n8n, JavaScript, OpenAI Responses API
INPUT	PDF, Excel and scanned documents in German, Czech and Polish
OUTPUT	ERP import files (order confirmations, delivery notes) + a review file
DEMO	4-minute video (link in the YouTube description and on my CV)

All companies and documents in the test set are fictional. They reproduce the structure and traps of real supplier documents.

01 — THE PROBLEM

45 seconds a line, by hand

A car-parts distributor's purchasing team typed every supplier document into the ERP by hand, about 45 seconds per line. Each supplier sends its own format, and the team has to copy order confirmations (what the supplier promises to deliver, at what price and when) and delivery notes (what actually shipped) into fixed Excel import templates.

What makes it hard

A 35–40 suppliers, each with its own layout, in German, Czech and Polish.

B Three file kinds: digital PDFs, Excel exports and scanned paper.

C Units that don't match the ERP: a supplier confirms "2 boxes of 10 x 1 l", but the ERP needs 20 pieces.

D Missing or hidden data: one supplier never prints its own name; another hides the order number in a free-text note.

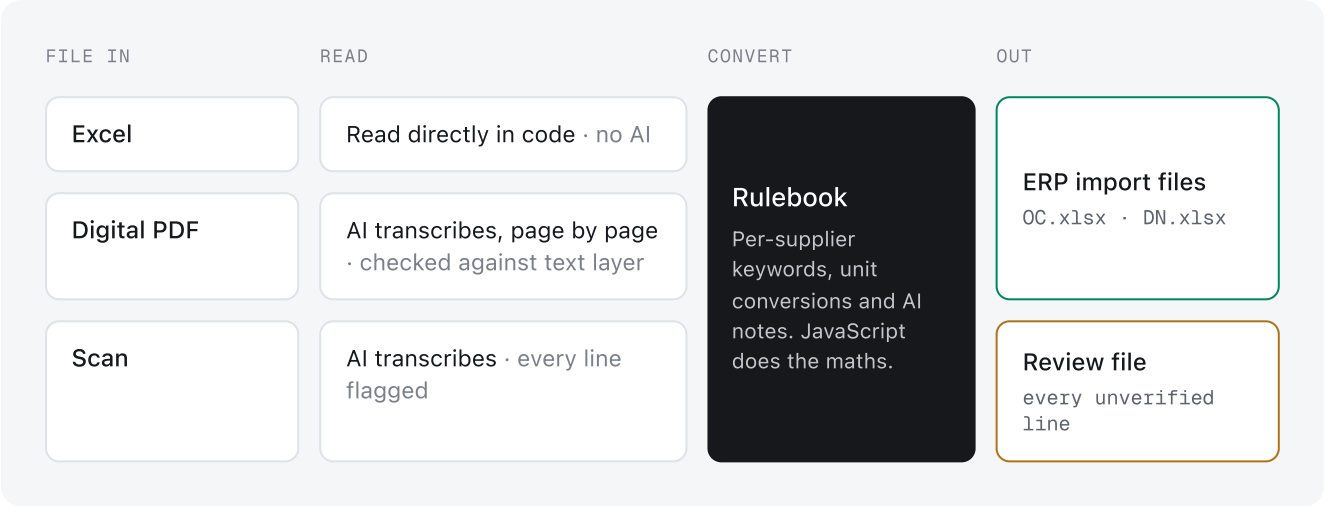
E Long documents: one delivery note runs to 14 pages and 395 lines across 3 purchase orders.

! **We can't ask suppliers to change. The automation has to work with what they send.**

I built the automation from the client's real documents. The client then decided not to go ahead, so I finished it as a portfolio case on a fictional test set that reproduces the same formats.

02 — THE APPROACH

Three roads, one rulebook, two outputs



Three design choices carry the workflow:

01

The AI copies, the code calculates.

The model transcribes values exactly as printed, into a strict JSON Schema. JavaScript does every conversion, so a calculation the AI never makes is one it can't get wrong.

02

Only PDFs and scans use AI.

Excel files are read directly in code. When a document arrives as both PDF and Excel, only the Excel is read.

03

One Rulebook.

Each supplier's identifying words, document-type keywords, conversion rules and notes for the AI live in one config node. Adding a supplier means adding an entry, not building new nodes.

03 — THE FAILURE, AND THE FIX

A green run that lost 374 lines

21 / 395

lines returned from the 14-page delivery note

The most useful moment in testing was a silent failure. The 14-page delivery note came back with only the lines from page 1: 21 of 395. The run still finished green and reported no problem, so 374 lines would have gone missing without anyone noticing.

Two changes fixed it

1 **Long PDFs are read one page at a time.** Any PDF longer than 4 pages (a setting in the Rulebook) is sent to the AI as separate page requests and joined back together in code.

2 **Every page is checked against its own text.** Code finds every product code printed on the page, using a pattern per supplier, and compares the two lists. A code on the page but missing from the answer means a skipped line. A code in the answer but not on the page means an invented one. Both go to the review file, naming the codes.

Scans have no text layer to check against, so every line read from a scan is flagged for a person to check visually.

An automation that can be wrong must also be able to tell you when it is wrong.

Every uploaded file ends up either in the import files or in the review file, with a reason.

04 — RESULTS

Every line, every run

Tested against a hand-checked expected answer for all 12 documents, the workflow read every line correctly across repeated runs.

TEST	RESULT
Order confirmation lines	**73 of 73 correct**
Delivery note lines	**425 of 425 correct**
Upload only the Excel files	Works; only Excel lines returned
Upload only the PDFs	Works; the 14-page note is read page by page
Simulated "lazy" AI that skips lines	Caught; missing codes listed in the review file
AI cost	≈ $0.00008 per line, ≈ $0.04 per full run
Manual typing, for comparison	≈ 45 s per line, ≈ 6 hours for 498 lines

05 — WHAT PRODUCTION WOULD NEED

From test set to rollout

Supplier entries

One Rulebook entry per supplier, built from 2–3 sample documents each, then tested against a known answer.

Intake

A different intake, such as a shared mailbox or folder instead of the upload form; everything after the file split stays the same.

Pilot

A short pilot on live documents with the purchasing team checking the review file, before any line goes into the ERP unchecked.

Data & hosting

Data agreement and hosting choice, since supplier documents are sent to an AI provider.